

PSEUDOCÓDIGO (I)

Programación

INTRODUCCIÓN A LA PROGRAMACIÓN



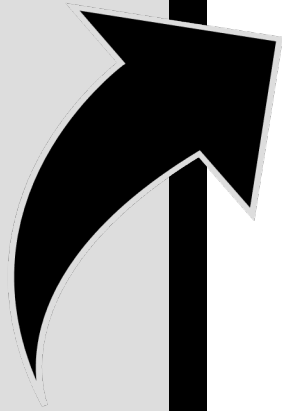
INTRODUCCIÓN A LA PROGRAMACIÓN

- Un ordenador **resuelve problemas**, procesa una información para obtener un resultado a partir de unos datos de entrada.
 - Los ordenadores resuelven los problemas mediante la **utilización de programas escritos por los programadores**.
 - **PROGRAMACIÓN** → proceso de dotar al ordenador de un método para resolver un problema tipo y entregar un resultado.
- Un programa está compuesto de 2 partes:
 - **Código** → conjunto de acciones a ejecutar, *instrucción*.
 - **Estructura de datos** → disposición de memoria elegida para almacenar los datos necesarios para resolver el problema, **variables**.



El ordenador necesita ser programado, es decir, introducir a su memoria instrucciones que la guíen en lo que tiene que hacer realizando en cada paso tareas muy básicas.

El proceso de programación se divide en las siguientes tareas



- 1. Comprensión del problema.*
- 2. Idear una solución que sea lo más general posible.*
- 3. Enunciar un algoritmo de resolución.*
- 4. Codificarlo en un lenguaje de programación.*
- 5. Implementarlo, hacer que la máquina pueda traducirlo a su propio código .*
- 6. Preparar un **lote de prueba**.*
- 7. Ejecutar el programa y validarlo mediante el **lote de prueba**.*

ALGORITMO

Conjunto ordenado y finito de operaciones (o pasos) que permiten resolver un problema.

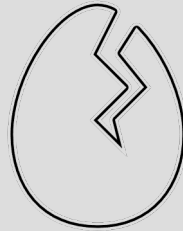
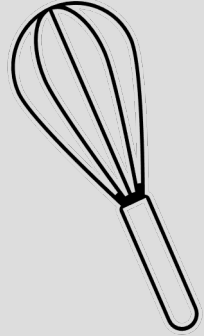
Cumplen las siguientes características:

- Tiene un número finito de pasos
- Acaba en un tiempo finito.
- Todas las operaciones deben estar definidas de forma precisa y sin ambigüedad.
- Puede tener varios datos de entrada y como mínimo un dato de salida.

ALGORITMO

EJEMPLO → Receta de cocina.

Tenemos unos pasos que hay que seguir en un orden y deben de estar bien definidos, tiene un tiempo finito y tiene unos datos de entrada (ingredientes) y una salida (el plato).



El algoritmo para freír un huevo podría ser:

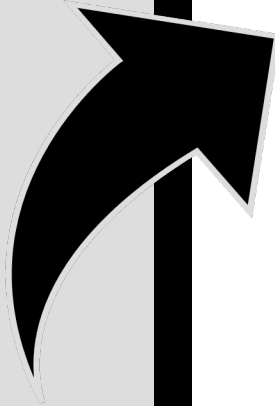
- Datos de entrada: Huevo, aceite, sartén, fuego.
- Datos de salida: huevo frito.
- Procedimiento:
 1. Poner el aceite en la sartén.
 2. Poner la sartén al fuego.
 3. Cuando el aceite esté caliente, cascar el huevo e introducirlo.
 4. Cubrir el huevo de aceite.
 5. Cuando el huevo esté hecho, retirarlo.

ALGORITMO COMPUTACIONAL

*La codificación de un algoritmo en un ordenador se denomina **programa**.*

Las acciones que serán permitidas en un algoritmo son las siguientes:

- Introducir un valor en una variable desde un dispositivo de entrada
- Mostrar en un dispositivo de salida un valor
- Almacenar un valor en una variable como resultado de una expresión
- Bifurcar el flujo de ejecución entre dos alternativas.
- Repetir parte del flujo normal de proceso de una manera condicional.
- Iterar, o repetir parte del flujo un número determinado de veces.
- Seleccionar un único flujo de proceso a ejecutar de un conjunto de flujos posibles.



DIAGRAMA

Representación gráfica de una proposición, de la resolución de un problema, de las relaciones entre las diferentes partes o elementos de un conjunto o sistema.

En programación se utilizan los diagramas de Flujo de Datos (DFD).

En los DFD, el flujo está indicado por una línea descendente que indica la secuencia de ejecución de las instrucciones.

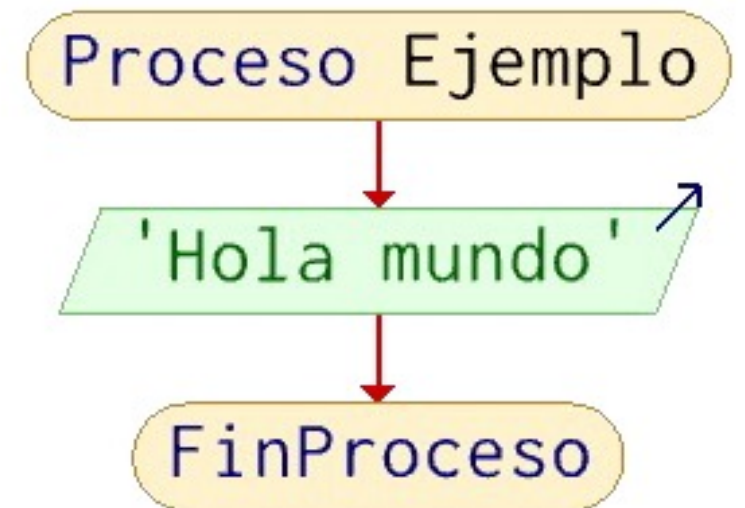
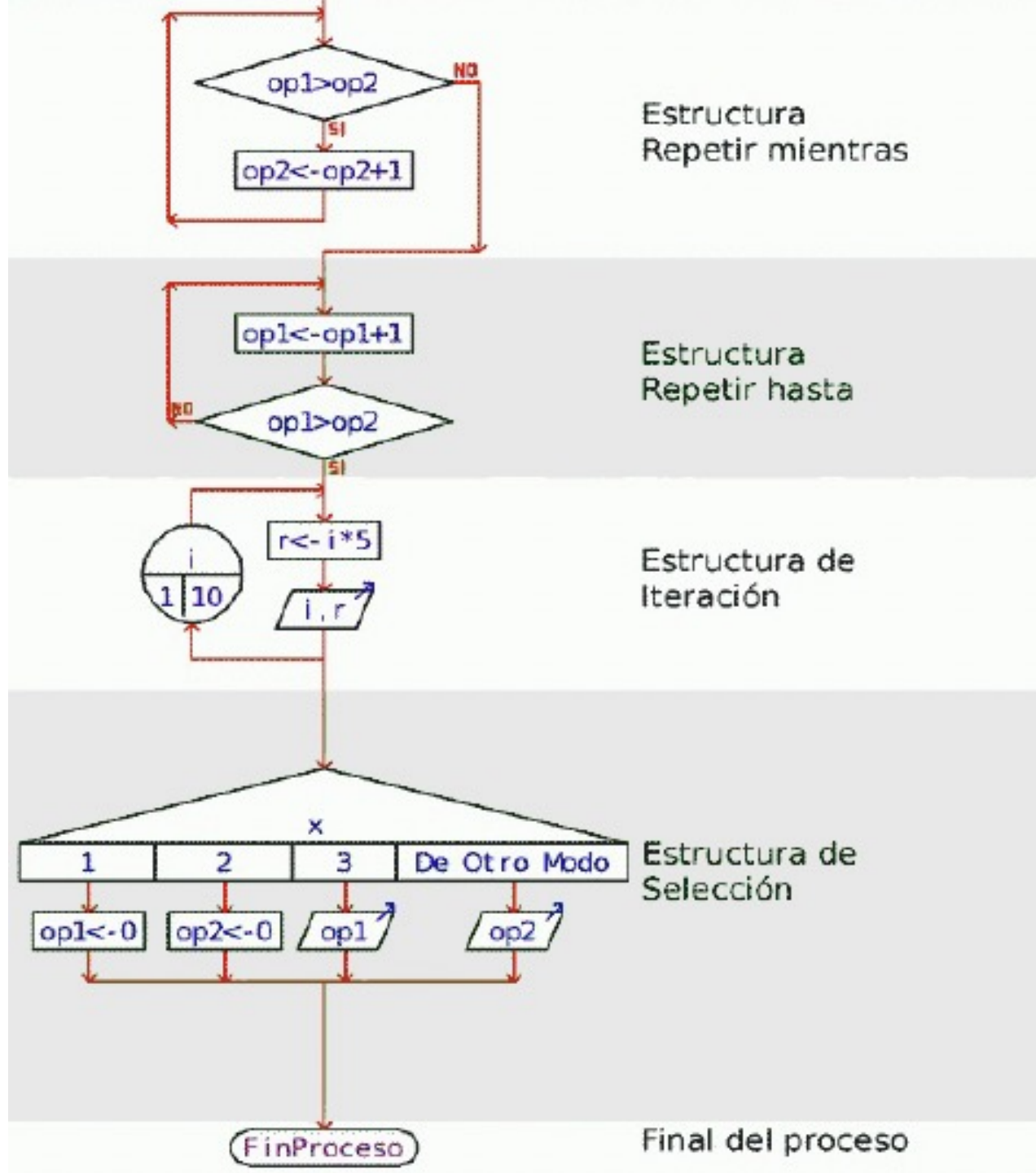
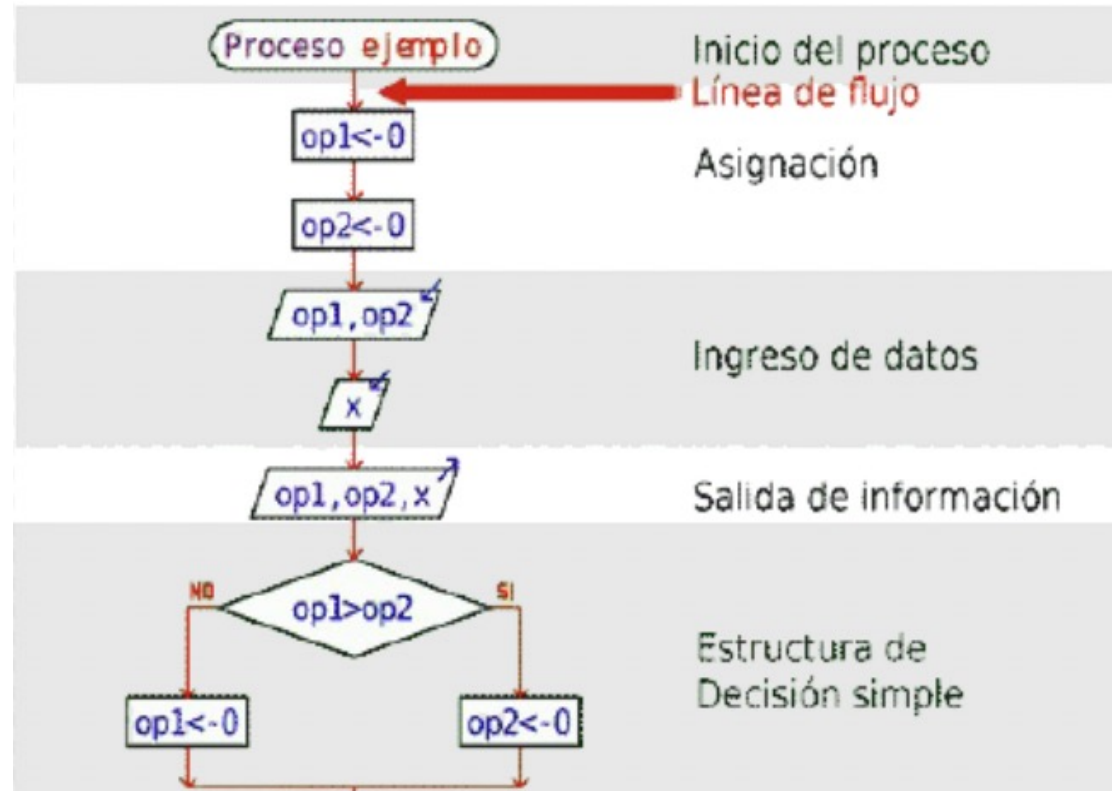


DIAGRAMA DE FLUJO DE PSEINT



PSEUDOCÓDIGO

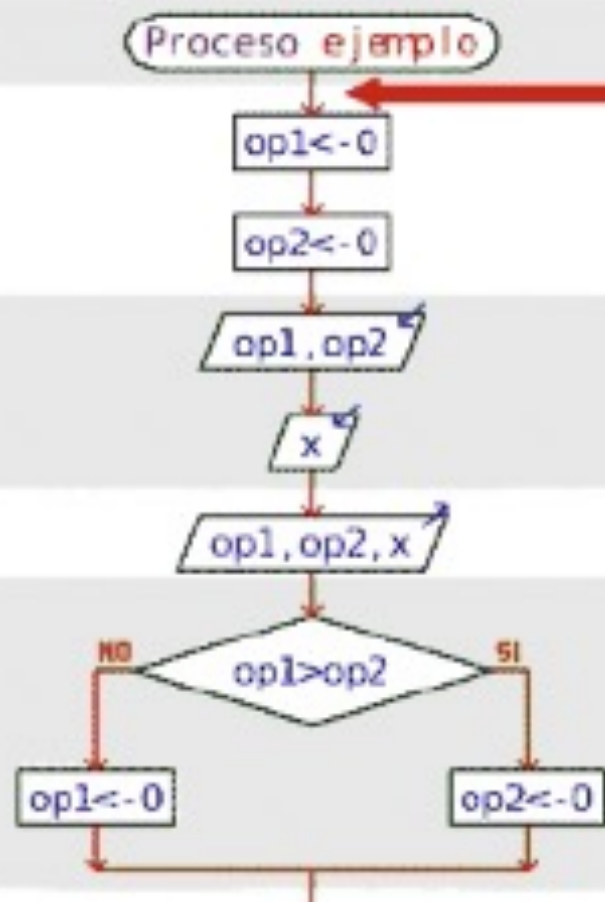
*Es otra forma de representar un algoritmo, utilizando una notación similar a un lenguaje de programación estándar, pero cuyas **palabras** claves están **escritas** en el lenguaje castellano.*

La idea es que se pueda ver el programa desde el punto de vista del programador el cual está acostumbrado a leer código fuente.

```
1  Proceso Ejemplo
2      Escribir "Hola mundo";
3  FinProceso
```

PSEUDOCÓDIGO DE PSEINT

Diagrama de Flujo de Datos



Inicio del proceso
Línea de flujo
Asignación

Ingreso de datos

Salida de información

Estructura de
Decisión simple

Pseudocódigo

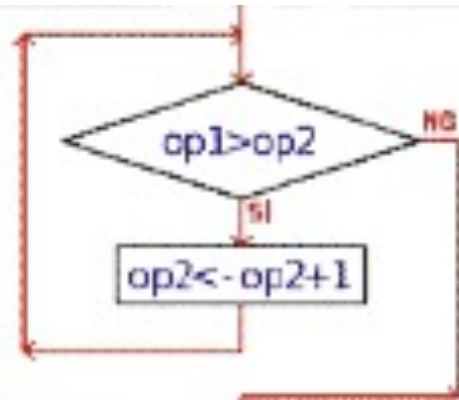
Proceso ejemplo

op1<-0
op2<-0

Leer op1, op2
Leer x

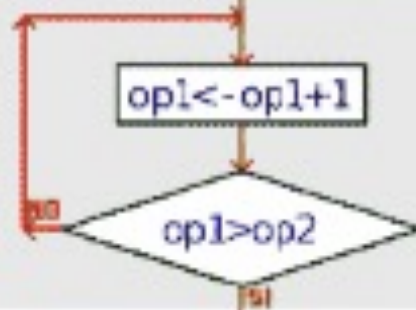
Escribir op1, op2, x
Si op1>op2 **Entonces**
 op2<-0
Sino
 op1<-0
FinSi

PSEUDOCÓDIGO DE PSEINT



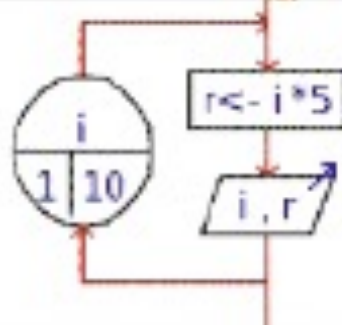
Estructura
Repetir mientras

```
Mientras op1 > op2 Hacer  
|   op2 <- op2 + 1  
FinMientras
```



Estructura
Repetir hasta

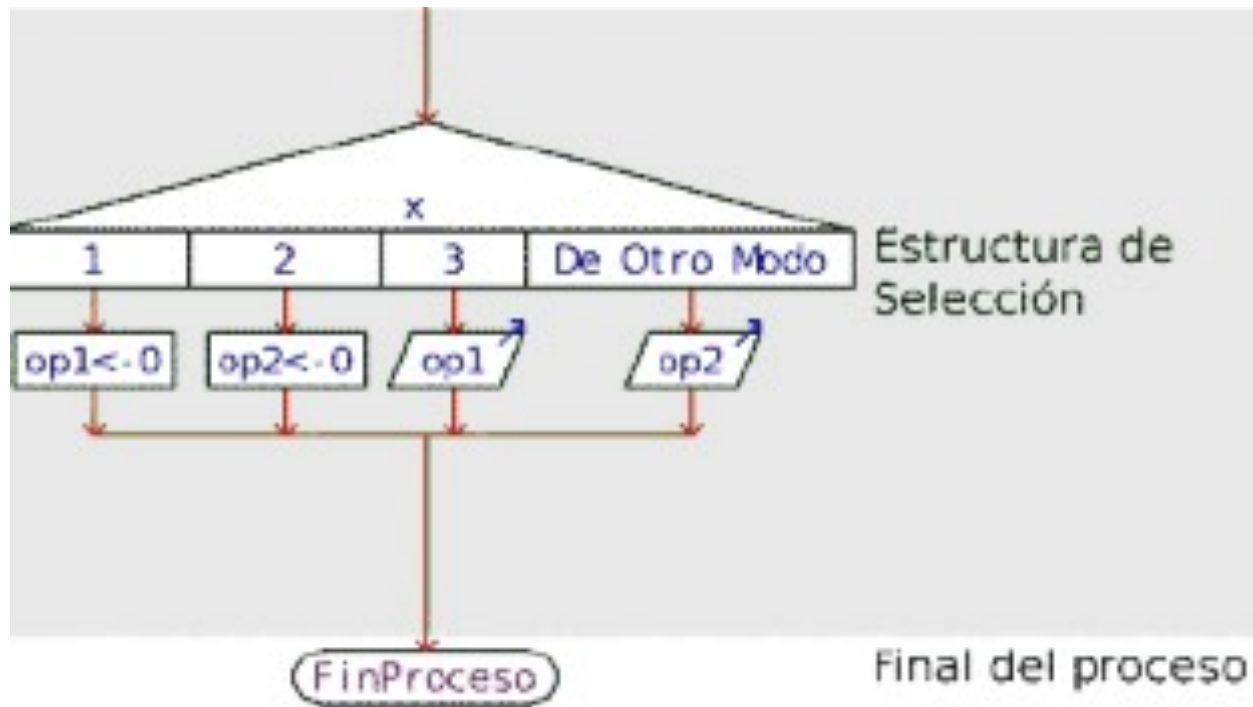
```
Repetir  
|   op1 <- op1 + 1  
Hasta Que op1 > op2
```



Estructura de
Iteración

```
Para i <- 1 Hasta 10 Hacer  
|   r <- i * 5  
|   Escribir i, r  
FinPara
```

PSEUDOCÓDIGO DE PSEINT



```
Segun x Hacer
1:      op1<-0
2:      op2<-0
3:      Escribir op1
De Otro Modo:
      Escribir op2
FinSegun
FinProceso
```

ESTRUCTURA DE UN PROGRAMA



ESTRUCTURA DE UN PROGRAMA

Un programa está compuesto de dos partes:

***código** → conjunto de acciones a ejecutar, **instrucción**.*

estructura de datos** → disposición de memoria elegida para almacenar los datos necesarios para resolver el problema, **variables

CÓDIGO → Texto que describe *la receta* que se debe aplicar para realizar una tarea compleja de procesamiento de datos.

- El ordenador ejecuta programas que traducen texto generado por el usuario (**código fuente**) a operaciones en código máquina (**código objeto**) y puede ejecutar cada acción descrita en cada oración.
- Estos programas traductores se denominan genéricamente programas **intérpretes**.
- Problema: los lenguajes naturales son poco precisos al momento de definir tareas.

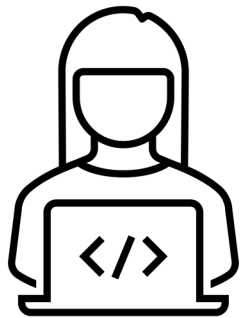
ESTRUCTURA DE UN PROGRAMA

Cuando programamos describimos paso por paso y con oraciones muy simples que debe hacer el ordenador para aplicar un algoritmo.

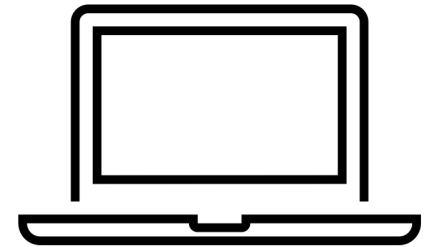
- Los programas intérpretes reconocen un lenguaje muy acotado (*que se basa en su amplia mayoría en el idioma inglés*) resumiendo el vocabulario normal a unas pocas palabras con un significado preciso.
- El objetivo es hacer que el texto que podamos crear no sea difícil de entender y traducir por no tener ninguna ambigüedad en su significado.
- Como todo texto, un programa no es más que un conjunto de oraciones.
- Un programa se compone de oraciones de tipo declarativas e imperativas.

ESTRUCTURA DE UN PROGRAMA

- La oración está sujeta a leyes sintácticas que son muy estrictas.
- Hay que realizar un análisis sintáctico de todo el texto o de cada oración indicando todos los errores sintácticos detectados para que los corrijamos.
- Cuando el programa no tiene errores sintácticos se comienza con la traducción a código máquina y posterior ejecución.



PROGRAMA FUENTE



TRADUCCIÓN Y
EJECUCIÓN

SINTAXIS

INSTRUCCIONES

Oraciones imperativas cuya estructura es siempre: “hacer tal cosa con tales datos”. El verbo indica la orden que queremos que se ejecute, lo llamaremos **orden** o **comando**.

- Leer
- Escribir
- ← (asignar)

SENTENCIAS

Oraciones declarativas que sirven para dividir la secuencia de ejecución en **bloques** que incluyan instrucciones u otras sentencias que se ejecutaran en forma grupal, delimitando donde comienza y donde termina este bloque.

- Proceso ... / finProceso
- Si ... Entonces / Sino /finSi
- Mientras ... Hacer / finMientras
- Repetir / Hasta que ...
- Para ... <- ... Hasta ... Hacer / finPara

ESTRUCTURA DE DATOS



ESTRUCTURA DE DATOS

Conjunto de datos que requiere el programa.

*Entendemos por **dato** de un programa todo aquello que pueda ser manipulado por las instrucciones.*

Sus atributos son:

- **Nombre:** el identificador del objeto.
- **Tipo:** conjunto de valores que puede tomar.
- **Valor:** elemento del tipo que se le asigna.

ESTRUCTURA DE DATOS

CONSTANTES

- Son objetos cuyo valor permanece **invariable** a lo largo de la ejecución de un programa.
- Una constante es la ***denominación de un valor concreto***, de tal forma que se utiliza su nombre cada vez que se necesita referenciarlo.
- *Por ejemplo:* $\pi = 3.14.1592$

IVA = 0,21

VARIABLES

- Son objetos cuyo valor **puede ser modificado** a lo largo de la ejecución de un programa.
- *Por ejemplo:* una variable para calcular el área de una circunferencia determinada, una variable para calcular una factura, etc.

CONSTANTES Y VARIABLES

Cada variable o constante debe poseer un nombre único, éste debe comenzar por una letra (vocal o consonante) y luego puede continuar con un conjunto de letras o números. Queda prohibido como parte del nombre cualquier símbolo y tampoco se admiten espacios.

Respecto a los tipos primitivos de datos, en pseudocódigo, existen 3:

- El tipo **lógico** o booleano, sólo dos posibles valores: verdadero o falso.
- El tipo **numérico**, puede operarse matemáticamente, por ejemplo, una medida, el resultado de un cálculo, etc.
- El tipo **alfanumérico**, no es matemático, por ejemplo, un nombre, una denominación, un código, etc. a los alfanuméricos también se los conoce como strings o cadenas de texto.

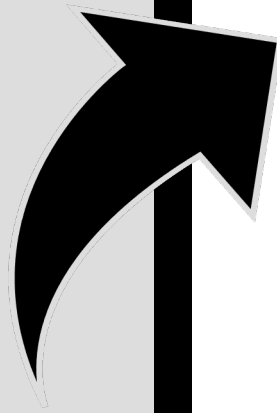
EXPRESIONES



EXPRESIONES

Consiste en el resultado de un cálculo.

Las expresiones según el resultado que produzcan se clasifican en:



- **Numéricas:** Producen resultados de tipo numérico. Se construyen mediante los operadores aritméticos.

Por ejemplo: $\pi * \text{sqr}(x)$ o $(2*x)/3$

- **Alfanuméricas:** Producen resultados de tipo alfanumérico. Se construyen mediante operadores alfanuméricos.

Por ejemplo: "Don " + "José"

- **Booleanas o lógicas:** Producen resultados de tipo Verdadero o Falso. Se construyen mediante los operadores relacionales y lógicos.

Por ejemplo: $a < 0$ o $(a > 1) \text{ and } (b < 5)$

EXPRESIONES

OPERADOR

- Es un símbolo que representa una operación entre datos.
- Algunos son muy conocidos como por ejemplo + y - (la suma y la resta), pero hay muchos más.

Una expresión está compuesta por datos afectados por operadores y/o funciones.

FUNCIÓN

- Es un proceso definido que se hace con uno o varios datos y del cual se obtiene un único resultado.
- Por ejemplo las funciones trigonométricas seno(), coseno(), etc.

OPERADORES

Pueden ser de los siguientes tipos: matemáticos o aritméticos, relacionales, lógicos y concatenación.

ARITMÉTICOS

- Operadores matemáticos. El resultado de una operación matemática es un tipo numérico.

OPERADOR	DEFINICIÓN
+	Suma
-	Resta
*	Multiplicación
^	Potencia
/	División
%	Resto de la división

Ejemplos:

EXPRESIÓN	RESULTADO
3 + 5 - 2	6
24 % 3	0
8 * 3 - 7 / 2	20.5

OPERADORES

Pueden ser de los siguientes tipos: matemáticos o aritméticos, relacionales, lógicos y concatenación.

RELACIONALES

- Operadores relacionales. El resultado es un tipo lógico ya que nos dice si la relación es verdadera o falsa

OPERADOR	DEFINICIÓN
<	Menor que
>	Mayor que
=	Igual que
>=	Mayor o igual que
<=	Menor o igual que
<>	Distinto que

Ejemplos:

EXPRESIÓN	RESULTADO
'A'<'B'	Verdadero
1<6	Verdadero
10<2	Falso

OPERADORES

Pueden ser de los siguientes tipos: matemáticos o aritméticos, relacionales, lógicos y concatenación.

LÓGICOS

- Operadores lógicos. El resultado es un tipo lógico siguiendo las reglas de las tablas de verdad del álgebra de Boole.

OPERADOR	DEFINICIÓN
No	Negación
Y	Conjunción
O	Disyunción

OPERADORES LÓGICOS

NO lógico (NOT) o negación

A	NOT A
V	F
F	V

O lógica (OR) o disyunción:

A	B	A OR B
V	V	V
V	F	V
F	V	V
F	F	F

Y lógica (AND) o conjunción:

A	B	A AND B
V	V	V
V	F	F
F	V	F
F	F	F

OPERADORES LÓGICOS

Ejemplos:

EXPRESIÓN	RESULTADO
$9 = (3 * 3)$	Verdadero
$3 <> 2$	Verdadero
$9 = (3*3) \text{ Y } 3 <> 2$	Verdadero Y Verdadero = Verdadero
$3 > 2 \text{ Y } 'B' < 'A'$	Verdadero Y Falso = Falso
$3 > 2 \text{ O } 'B' < 'A'$	Verdadero O Falso = Verdadero
$\text{No } ('A' < 'B')$	No Verdadero = Falso
$5 > 1 \text{ Y No } ('B' < 'A')$	Verdadero Y No Falso = Verdadero

OPERADORES

*Pueden ser de los siguientes tipos:
matemáticos o aritméticos,
relacionales, lógicos y
concatenación.*

CONCATENACIÓN

- Operador de concatenación. Es el único operador que permite operar entre si dos datos de tipo alfanumérico, el resultado es una cadena de texto que contiene las dos cadenas unidas una continuación de la otra.

Ejemplos:

EXPRESIÓN	RESULTADO
"Ana" + "López"	Ana López
"saca" + "puntas"	sacapuntas

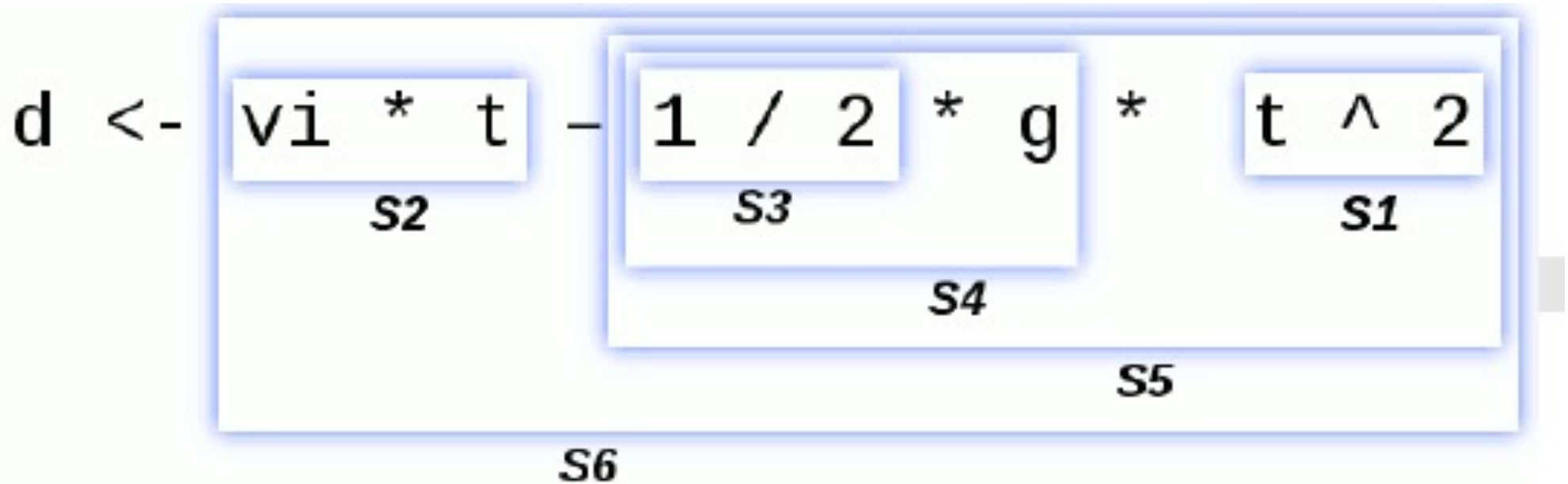
ORDEN DE EVALUACIÓN DE LOS OPERADORES

Las reglas de precedencia indican que operador va a tener mayor prioridad de resolución sobre otro, y si ambos tuvieran la misma prioridad, se resuelven de izquierda a derecha.

1. Paréntesis.
2. Potencia ^
3. Multiplicación, división y resto * / %
4. Sumas y restas + -
5. Concatenación +
6. Relacionales < <= > >= etc.
7. Negación NOT
8. Conjunción AND
9. Disyunción OR

ORDEN DE EVALUACIÓN DE LOS OPERADORES

Ejemplo:

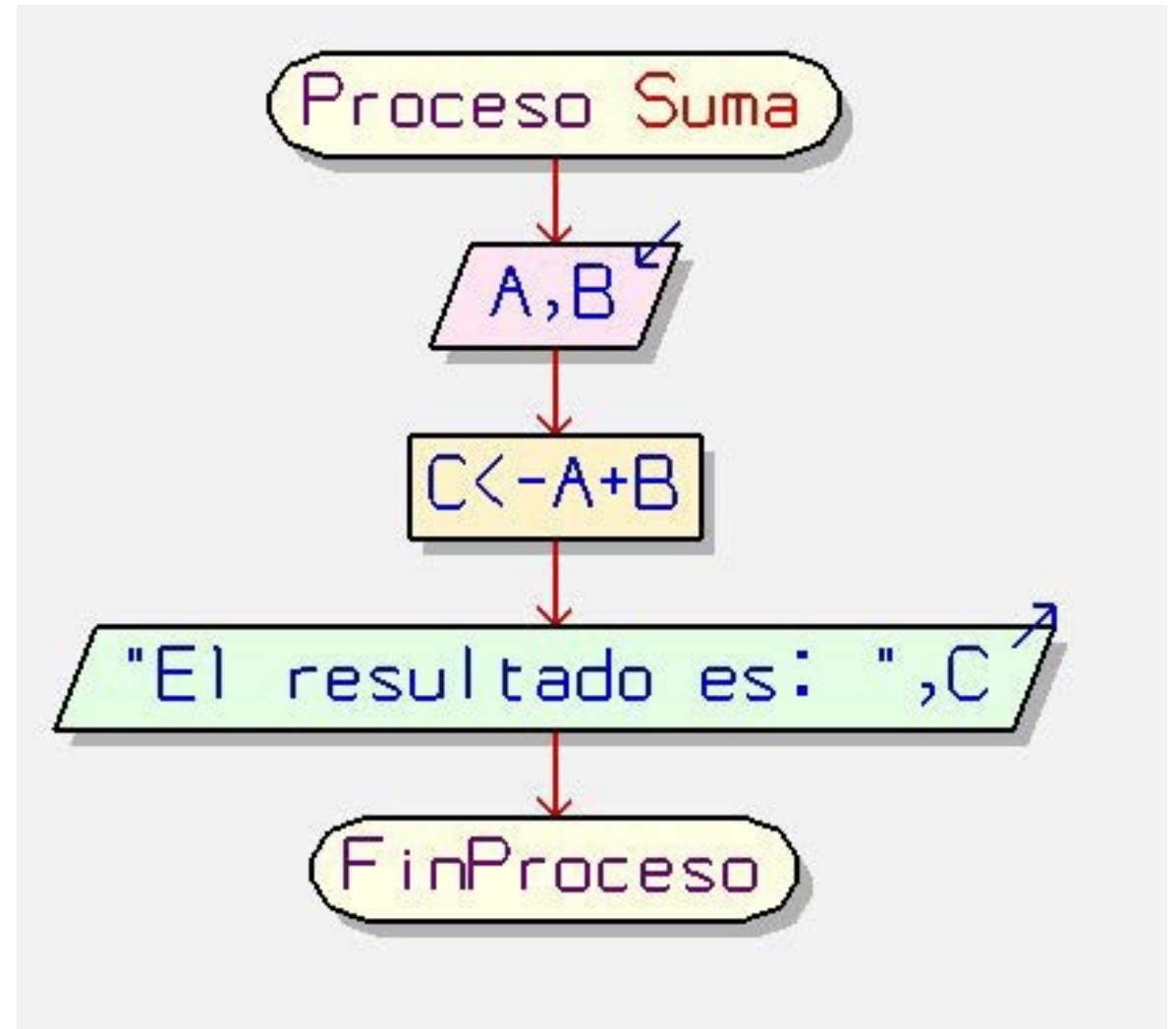


COMENZANDO A PROGRAMAR



ALGORITMO DE SUMA

Se necesita confeccionar un programa que sume dos números y nos muestre el resultado.



ALGORITMO DEL TRIANGULO

*Calcular el área y el
perímetro de un triángulo
conociendo el largo de los 3
lados.*

